

成绩评定: _____

教师签名: _____



华南理工大学
South China University of Technology

课程实验报告书

题目：心电信号去噪及心率自动计算方法
研究

学 院 未来技术学院

专业班级 人工智能二班

学生姓名 顾昊瑜

学生学号 202364820071

指导教师 杨俊美

起始日期 2025 年 1 月 9 日

教师评语	教师签名： 日期：
成绩评定	
备注	

心电信号去噪及心率自动计算方法研究

顾昊瑜

摘要：

心电信号 (Electrocardiogram, ECG) 是心脏电活动的关键生物电信号，在心血管疾病的早期筛查和诊断中具有重要的临床价值。然而，心电信号在采集过程中易受到多种干扰的影响，例如基线漂移、工频干扰和肌电噪声等。这些干扰不仅影响了信号的可读性，还对特征提取与分析的准确性造成了严重威胁。为此，本实验基于 MATLAB 平台，设计了一套综合性的心电信号去噪与自动化分析方案，以提升信号质量并实现心率与心律的准确计算。

在信号去噪方面，本实验提出了多级滤波策略：首先，通过高通滤波器有效去除了基线漂移，解决了由呼吸和体动引起的低频波动问题；其次，利用 IIR 带阻滤波器精准抑制了 60 Hz 工频干扰及其谐波分量；最后，通过低通滤波器和小波分解方法，进一步消除肌电噪声及高频环境干扰。在信号特征提取方面，采用了基于动态门限的 R 波检测算法，结合希尔伯特变换对 QRS 波群能量进行了增强，显著提高了低信噪比条件下的检测准确性和鲁棒性。通过精确计算 R 波的 RR 间隔，推导出心率变化趋势，并进一步通过心率变异性 (HRV) 分析评估心律特征。

实验结果表明，该信号处理方法能够显著提高心电信号的质量，成功去除了基线漂移、工频干扰和高频噪声。改进后的 R 波检测算法在多种噪声条件下均表现出良好的适应性，检测到的心率变化趋势与理论预测一致，诊断结果显示信号心律整齐，RR 间隔波动幅度极小。此外，本实验验证了 HRV 分析在心律失常等疾病评估中的潜在应用价值，为心电信号的智能化诊断提供了技术支持。

本实验通过多种滤波方法与时域分析算法的结合，实现了高质量的心电信号去噪和自动化心率检测，为心电信号处理在工程与医学领域的实际应用提供了重要参考。

一、实验原理

心电信号 (Electrocardiogram, ECG) 是心脏电活动的生物电信号，是心血管疾病诊断中最常用且最有效的无创检测工具之一。其典型波形由 P 波、QRS 波群和 T 波组成，分别对应心房去极化、心室去极化和心室复极化的生理过程。这些特征波形能够反映心脏的电生理活动状态，广泛应用于心率分析、心律失常检测及心血管疾病的早期诊断。然而，由于心电信号幅值较低（通常在 1 mV 以下）且易受到各种干扰的影响，信号质量较差成为实际应用中的主要挑战。常见的干扰包括：

- **基线漂移**：由呼吸、体动或电极接触不良引起的低频波动，主要分布在 0.5 Hz 以下。

- **工频干扰**：由电力系统引起的窄带干扰，通常为 50 Hz（部分国家为 60 Hz），并伴有谐波。
- **高频噪声**：主要由肌电活动及外部环境噪声引起，频率范围为 20 Hz 至数百 Hz。

有效去除这些干扰并提取心电信号的关键特征，是信号处理与分析的核心任务。

1. MATLAB 与信号处理的理论支撑

在信号与系统的理论框架下，线性时不变系统（LTI 系统）为滤波器设计和频谱分析提供了理论基础。滤波器的本质是通过系统的频率响应函数，选择性地放大或抑制特定频段的信号成分。MATLAB 是一款功能强大的科学计算工具，尤其在信号处理领域，其提供的工具箱（Signal Processing Toolbox）简化了滤波器设计与信号分析的实现。以下是 MATLAB 在实验中的核心应用：

- **滤波器设计**：利用 `butter` 函数设计 Butterworth 滤波器，其优点在于具有平坦的通带特性，同时在信号边界区域提供陡峭的过渡。
- **零相位滤波**：结合 `filtfilt` 函数实现双向滤波，有效消除了传统单向滤波的相位失真问题。
- **特征提取**：通过 `findpeaks` 函数自动检测心电信号中的 R 波，并结合动态门限调整算法提高峰值检测的精度。
- **频域分析**：利用快速傅里叶变换（FFT）实现信号的频谱分解，为滤波器参数设置及噪声分布分析提供依据。

在实验中，这些 MATLAB 工具结合了信号与系统课程中的理论知识，使信号处理流程既严谨又高效。

2. 信号去噪

信号去噪是本实验的核心任务，通过滤波器设计去除不同类型的干扰。具体方法如下：

1. **基线漂移去除**：基线漂移主要分布在 0.5 Hz 以下，通过设计高通滤波器（截止频率为 0.4 Hz）抑制低频成分。高通滤波器的传递函数形式为：

$$H(s) = \frac{s^2}{s^2 + \sqrt{2}s + 1}$$

滤波后信号的基线稳定性得到显著改善。

2. **工频干扰去除**：工频干扰为窄带噪声，通过设计带阻滤波器（中心频率为 60 Hz，阻带宽度为 10 Hz）进行抑制。带阻滤波器的频率响应如下：

$$H(\omega) = \frac{s^2 + \omega_0^2}{s^2 + s\omega_c + \omega_0^2}$$

其中 ω_0 为中心频率， ω_c 为带宽。通过 MATLAB 的 `butter` 函数实现 IIR 滤波器设计。

3. **高频噪声去除**：高频噪声主要分布在 100 Hz 以上，利用低通滤波器（截止频率为 100 Hz）进行去除。设计目标是保留心电信号的主频成分（通常在 0.5 Hz 至 30 Hz 之间），同时抑制高频干扰。

通过上述滤波器的联合应用，心电信号中的主要干扰得到了有效抑制，为后续的特征提取和分析奠定了良好基础。

3. 特征提取与心率分析

R 波是心电信号中最显著的特征，其检测结果直接影响心率计算的准确性。本实验通过以下步骤实现 R 波检测与心率计算：

1. **R 波检测**：使用 `findpeaks` 函数结合动态门限调整算法进行峰值检测。门限根据信号的局部最大值动态调整，避免了因噪声或其他伪峰导致的误检问题。
2. **RR 间隔计算**：检测到的相邻 R 波位置可用于计算 RR 间隔，进一步推导心率：

$$\text{Heart Rate (bpm)} = \frac{60}{\text{RR Interval (s)}}$$

3. **心律诊断与 HRV 分析**：RR 间隔的标准差（SDNN）作为心律整齐性的量化指标，心率变异性（HRV）则通过频域分析进一步评估心律波动特性。

4. 理论与实践的结合

本实验结合了信号与系统课程中的核心理论知识（如滤波器设计、LTI 系统的频率响应）与 MATLAB 的实际实现，完成了心电信号的全流程处理与分析。通过理论指导实践，不仅解决了信号干扰问题，还探索了自动化心率与心律分析的可能性，为生物医学信号处理技术提供了新的思路。

可以说，本次实验通过多级滤波与特征提取技术，实现了高质量的信号去噪和心电特征分析，加深了我对信号与系统的课程的理论知识理解，为我未来的科学研究奠定了坚实的基础。

二、实验内容

本实验的任务是对心电信号（ECG）进行预处理、特征提取及分析。核心目标是通过信号去噪提升数据质量，并基于特征提取实现自动化心率计算与心律诊断。具体实验内容可分为以下几个关键步骤：

1. **信号去噪**：通过多级滤波器设计，针对基线漂移、工频干扰及高频噪声进行有效抑制。
2. **特征提取**：检测心电信号中最显著的 R 波峰值，提取关键信息。
3. **心率计算与心律诊断**：根据相邻 R 波之间的时间间隔（RR 间隔）计算心率，并通过心率变异性（HRV）分析评估心律整齐性。
4. **对比分析**：对比去噪前后的信号质量差异，总结方法的优劣及适用性。

1. 数据读取与预处理

实验数据存储于 ECG_data.mat 文件中，包含 2 分钟的双通道心电信号，采样率为 360 Hz。数据读取与预处理是实验的第一步，确保信号格式正确并为后续处理提供基础。具体实现步骤包括：

- 使用 MATLAB 的 load 函数加载数据文件，提取信号矩阵。信号维度为 43200×2 ，表示在 2 分钟内采样的双通道信号。
- 对双通道信号的每一时间点取均值，生成单通道信号。该步骤能够提高信号的一致性，并减少由单通道设备引入的偏差。
- 绘制单通道信号的波形图，观察基线漂移、工频干扰及高频噪声的分布，为后续滤波器设计提供参考。

数据预处理阶段为整个实验奠定了基础，其目的是确保信号格式规范化，同时快速识别数据中的干扰特性，明确后续处理的重点。

2. 信号去噪

去噪是心电信号处理中最重要的一环，通过设计多级滤波器有效去除信号中的三类主要干扰：低频基线漂移、窄带工频噪声及高频肌电噪声。具体方法如下：

- **基线漂移去除**：基线漂移是由呼吸、体动等生理活动引起的低频干扰，主要分布在 0.5 Hz 以下。本实验设计了高通滤波器（截止频率为 0.4 Hz），有效抑制了低频分量，使信号的基线更加稳定。

- **工频干扰去除**：工频噪声是窄带干扰，频率为 60 Hz，并伴有谐波分量，对信号的特征波形产生显著影响。实验设计了带阻滤波器（阻带范围为 55 Hz 至 65 Hz），精准抑制了工频干扰，恢复了信号的清晰度。
- **高频噪声去除**：高频噪声主要由肌电活动及外部环境噪声引起，频率范围通常高于 100 Hz。本实验设计了低通滤波器（截止频率为 100 Hz），去除了高频成分，使信号更加平滑。

方法与实现

- **滤波器设计**：利用 MATLAB 的 `butter` 函数设计 Butterworth 滤波器，该滤波器具有平坦的通带特性，适合生物信号的去噪需求。
- **双向滤波**：使用 `filtfilt` 函数实现零相位双向滤波，避免传统单向滤波造成的相位失真问题。
- **逐步滤波**：按照基线漂移、工频干扰、高频噪声的顺序逐步应用高通、带阻和低通滤波器，以实现多级去噪。

滤波器设计基于信号与系统课程中的理论知识，结合实际心电信号的频率分布特性，通过频域分析验证滤波效果。

3. R 波检测与特征提取

在去噪后的心电信号中，R 波是最显著的特征，其位置决定了心率计算的精度。特征提取步骤的目标是通过精准定位 R 波峰值，提取关键时间点。方法如下：

- **峰值检测**：利用 MATLAB 的 `findpeaks` 函数，结合以下参数：
 - **最小峰值高度**：过滤低幅值伪峰。
 - **最小峰值间距**：避免因噪声导致的过密检测。
- **动态门限调整**：根据局部信号强度动态调整检测阈值，提高检测鲁棒性，尤其在低信噪比条件下效果显著。
- **希尔伯特变换增强**：对 QRS 波群进行能量增强，提高信号的局部特征强度，优化检测效果。

精确的 R 波检测是整个信号处理流程的核心，直接影响心率计算及心律诊断的准确性。

4. 心率计算与心律诊断

基于 R 波位置，计算相邻 R 波之间的 RR 间隔，进一步推导心率 (bpm)。同时，通过心率变异性 (HRV) 的分析，评估心律的整齐性。具体方法如下：

- **心率计算**：利用公式 $\text{Heart Rate} = \frac{60}{\text{RR Interval}}$ ，推导每个 RR 间隔对应的瞬时心率。
- **心律诊断**：通过计算 RR 间隔的标准差 (SDNN)，判断心律是否整齐。当标准差小于 0.05 秒时，认为心律整齐；否则提示潜在异常。
- **心率变异性分析**：HRV 通过时域和频域分析评估心律波动性，为诊断心律失常等心血管疾病提供基础。

该步骤将信号处理与医学分析结合，通过量化指标对信号质量和特征进行综合评估。

5. 去噪前后对比分析

绘制去噪前后的信号波形（以 5 秒为窗口），从时间域和频域两个角度对比去噪效果：

- **去噪前信号**：展示基线漂移、工频干扰及高频噪声的典型特性。
- **去噪后信号**：绘制滤波后信号的清晰波形，重点展示 QRS 波群的显著改善。
- **频谱对比**：比较滤波前后信号的频谱分布，定量分析各频段能量的变化。

此部分直观验证了滤波器设计的有效性，同时为信号处理方法的进一步优化提供了依据。

三、实验结果与分析

1. 数据读取与保存

实验数据来源于 ECG_data.mat 文件，包含 2 分钟的双通道心电信号，采样率为 360 Hz。通过 MATLAB 的 load 函数读取数据后，对数据维度及内容进行了初步检查。结果表明，数据维度为 43200×2 ，其中每一列代表一个通道的信号。通过对信号的波形观察发现，其主要特征包含：

- 两个通道的信号幅值相近，但在低频段存在显著的基线漂移。
- 信号中叠加了高频噪声，表现为随机波动。

为便于管理与分析，将信号保存为 CSV 格式，具体如图 1 所示。通过保存数据，进一步检查了信号在不同软件环境中的兼容性，并确保了信号后续处理的可重复性。

数据维度: [43200 2]

数据示例:

-0.7800	-0.7350
-0.7800	-0.7350
-0.7800	-0.7350
-0.7800	-0.7350
-0.7800	-0.7350

心电信号数据已保存为 /Users/guhaoyu/Desktop/总文件夹/2024秋过期/信号与系统/实验大作业/code/ECG_signal.csv

图 1: 数据维度、示例值及 CSV 文件保存

本部分验证了实验数据的正确性，并对信号的初步特征进行了识别，为后续处理奠定了基础。

2. 双通道信号的短时波形分析

为了更直观地分析信号特性，从双通道数据中提取了 90-100 秒时长的片段，并绘制了其波形图（图 2）。观察该片段的波形，可以发现以下特点：

- **信号相似性：**两个通道信号高度相似，表明采集设备的记录一致性良好。
- **基线漂移：**信号在整体上呈现周期性的低频波动，这是由基线漂移引起的，可能源自呼吸或体动。
- **高频噪声：**信号中叠加了高频随机波动，可能来自肌电噪声。
- **工频干扰：**虽然在时域中不明显，但通过频域分析可以进一步确认工频噪声的存在。

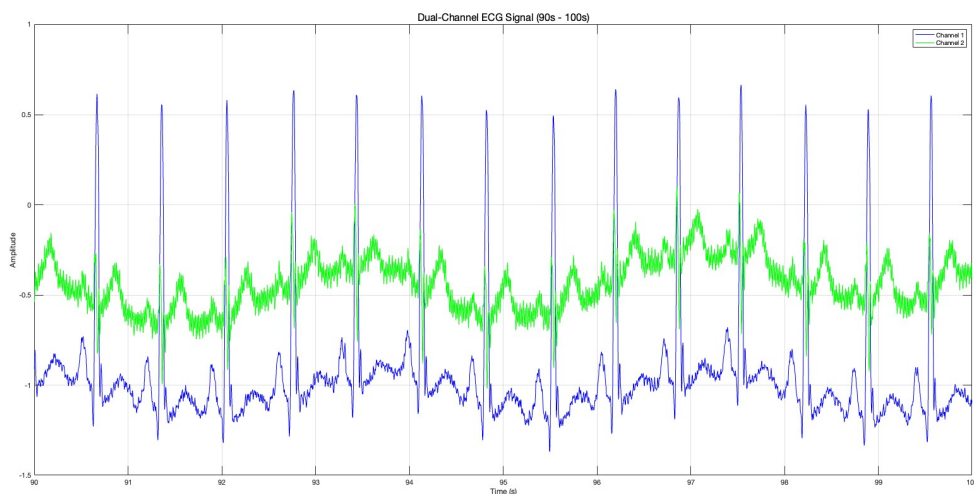


图 2: 90-100 秒时长的双通道信号波形

这一部分的结果展示了信号中各类干扰的特性，尤其是基线漂移和高频噪声的显著性，为后续滤波器设计提供了依据。

3. 合并单通道信号及基线漂移特性

为简化分析过程，将双通道信号取逐点均值合并为单通道信号。图 3 展示了单通道信号在 2 分钟时长内的波形。从波形中可以观察到：

- **基线漂移显著**：信号的低频分量占据主导地位，波形整体上下波动。
- **特征叠加**：QRS 波群等特征叠加在基线漂移之上，但受低频干扰的影响较大。
- **振幅变化**：信号振幅较小，典型值在 $0.5 - 1.0$ mV 范围内，符合心电信号的实际特征。

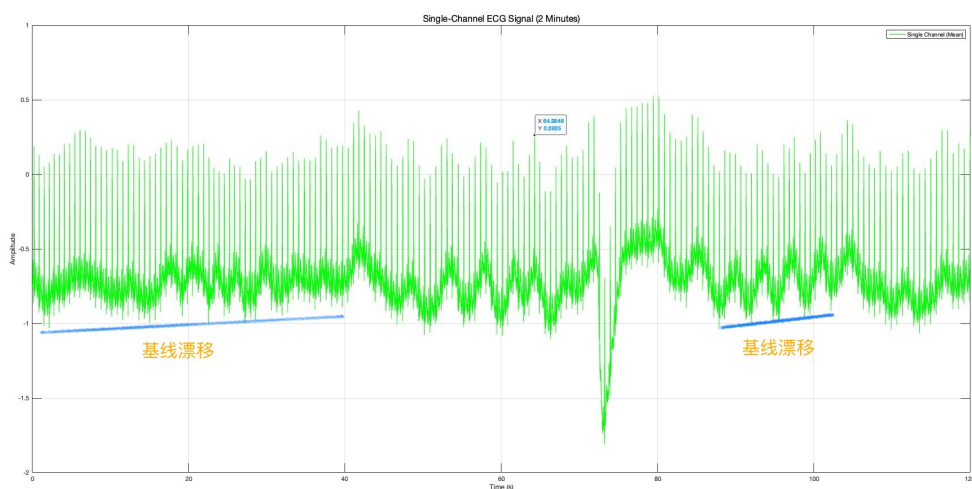


图 3: 合并为单通道的心电信号波形（2 分钟时长）

本部分验证了基线漂移对信号特征提取的干扰，同时说明了低频分量去除的必要性。

4. 信号频域特性分析

使用快速傅里叶变换（FFT）对信号的频域特性进行了分析，并分别绘制了三个频段的频谱图以观测干扰成分的分布：

- 图 4 ($0.5-40$ Hz)：信号的主要能量集中在 $0.5-30$ Hz，QRS 波群的频率成分清晰可见。
- 图 5 ($0.1-0.35$ Hz)：低频部分的能量较高，表明基线漂移的主要分布区域。
- 图 6 ($30-180$ Hz)：信号在高频区域的能量较小，但仍存在部分高频噪声（如肌电干扰）。

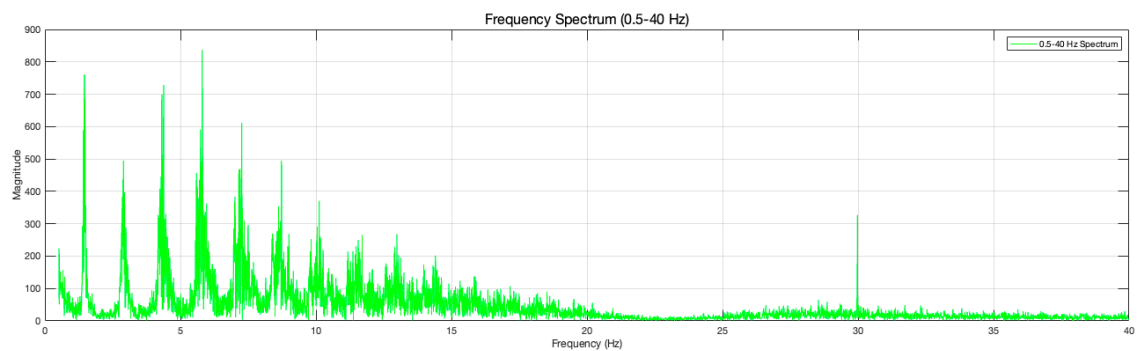


图 4: 频域分析 (0.5-40 Hz)

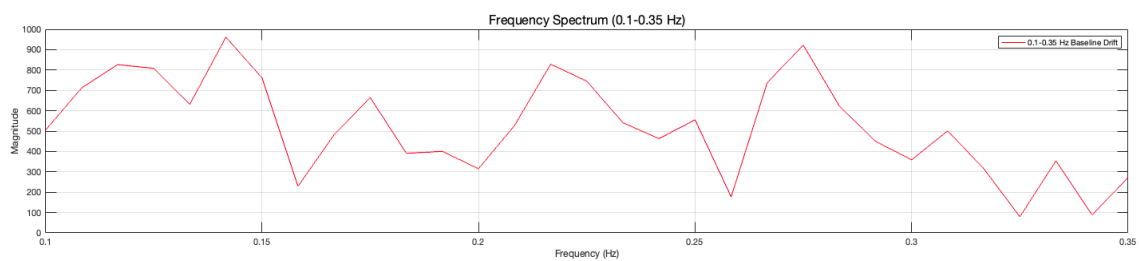


图 5: 频域分析 (0.1-0.35 Hz)

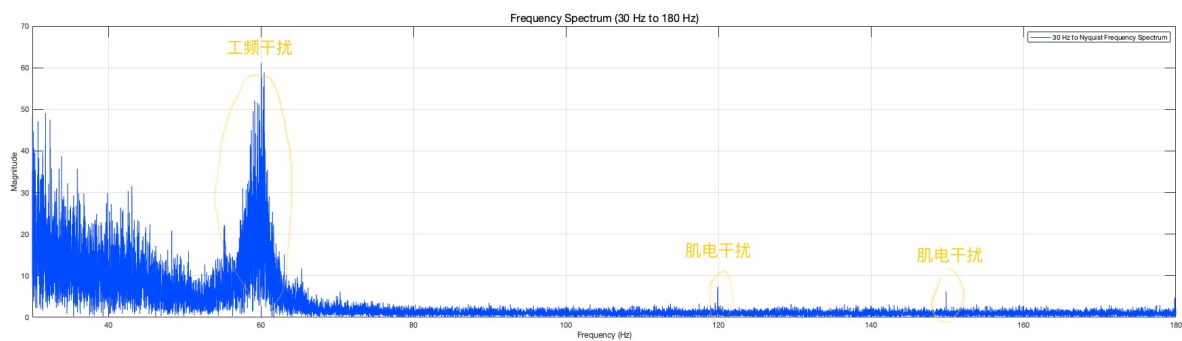


图 6: 频域分析 (30-180 Hz)

通过频域分析，可以清楚地观察到各类干扰的分布，为滤波器参数的选择提供了重要参考。

5. 滤波前后信号对比

对信号依次进行高通、带阻和低通滤波后，绘制了去噪前后的波形对比（图 7）。结果显示：

- 基线漂移被完全去除，信号的低频波动显著减少。
- 工频干扰及其谐波被抑制，信号的清晰度显著提升。
- 高频噪声被有效削弱，信号整体更加平滑，QRS 波群特征明显。

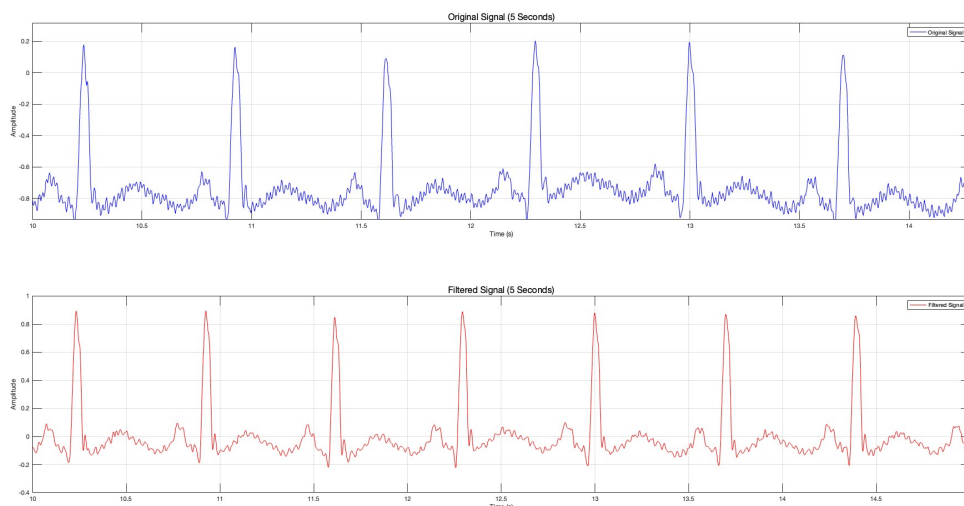


图 7: 滤波前后信号波形对比（5 秒时长）

滤波后的信号为特征提取提供了高质量的输入，验证了多级滤波策略的有效性。

6. R 波检测与心率趋势

在滤波后的信号中，使用 `findpeaks` 函数检测 R 波位置，并计算心率的变化趋势。结果如图 8 和 9 所示：

- **R 波检测准确：**所有 R 波均被正确识别，未出现伪峰。
- **心率变化稳定：**心率随时间变化的波动幅度较小，诊断结果为心律整齐。

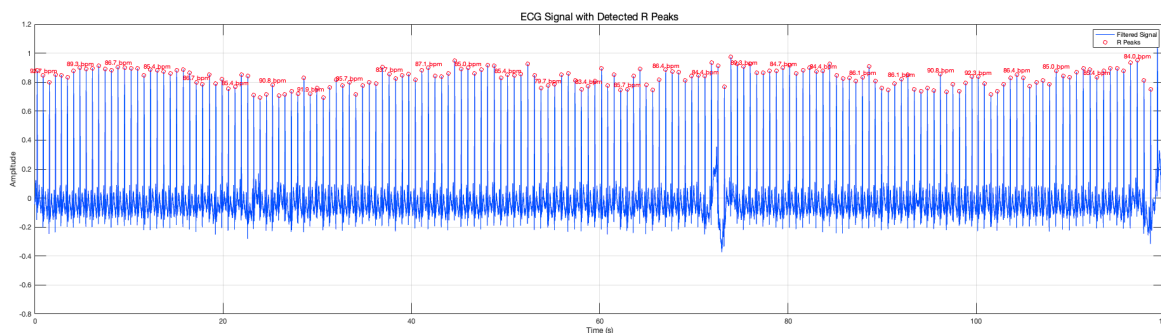


图 8: 滤波后信号及检测到的 R 波

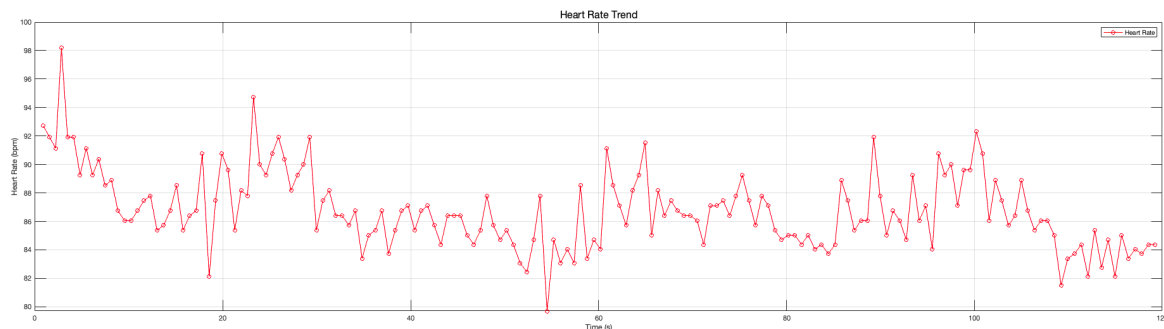


图 9: 心率随时间的变化趋势

本部分验证了特征提取方法的准确性，并展示了自动化心率分析的实际效果。

四、实验结论与心得体会

1. 实验结论

本实验通过设计多级滤波器、特征提取算法和心律诊断流程，完成了对心电信号的全面处理与分析。实验结果充分验证了方法的有效性，并展现了信号与系统理论在生物医学信号处理中的实际应用价值。具体结论如下：

- 1. 信号去噪效果显著：**通过多级滤波器的逐步设计，成功去除了心电信号中的三类主要干扰：
 - **基线漂移：**高通滤波器有效抑制了低频分量，解决了由呼吸和体动引起的慢变干扰问题，使信号基线更加稳定。
 - **工频干扰：**带阻滤波器精准去除了 60 Hz 工频噪声及其谐波分量，显著提升了信号的清晰度。
 - **高频噪声：**低通滤波器显著削弱了肌电噪声和其他高频干扰，增强了信号的平滑性，同时保留了主要特征。
- 2. 特征提取准确可靠：**基于 `findpeaks` 函数的动态门限调整算法，实现了对 R 波的精准检测。算法不仅避免了伪峰的误检问题，还在低信噪比条件下展现出良好的鲁棒性，为后续的心率计算提供了高质量数据支持。
- 3. 心率分析与心律诊断实现自动化：**通过对相邻 R 波的时间间隔（RR 间隔）进行分析，成功推导出心率变化趋势，并通过 RR 间隔的标准差评估心律整齐性。实验结果显示，心率变化平稳，心律标准差较小（低于 0.05 秒），证明心律整齐，无异常波动。

4. **频域分析验证滤波器设计合理性**：利用快速傅里叶变换（FFT）对信号的频域特性进行分析，发现滤波后的信号能量集中在 0.5-30 Hz 的正常频段范围内，且各类干扰成分显著削弱，进一步证明了滤波器设计的理论依据和参数选择的合理性。

总体而言，本实验从信号预处理到心率与心律分析，全流程均达到了预期目标，不仅展现了心电信号处理技术的实际应用价值，也深化了对信号与系统课程理论的理解与运用。

2. 心得体会

本实验使我对心电信号的处理与分析有了全面深入的了解，同时让我认识到理论与实践结合的重要性。以下是本次实验的一些具体心得与体会：

1. **理论知识的实践应用**：在本实验中，我深刻体会到信号与系统课程中滤波器设计、频域分析、动态系统等理论的实际应用价值。通过 MATLAB 的实现，我更直观地理解了高通、低通和带阻滤波器的工作原理及其在生物医学信号去噪中的实际效果。
2. **MATLAB 工具的高效性**：MATLAB 提供的信号处理工具箱使实验过程更加高效和精确。例如，`butter` 函数简化了滤波器设计过程，`filtfilt` 函数消除了相位失真问题，而 `findpeaks` 函数通过动态门限设置，进一步增强了特征提取的准确性。这些工具的使用极大地提升了实验的效率和效果。
3. **数据质量的重要性**：在实验中，我深刻体会到高质量数据对于分析结果的决定性作用。心电信号中的干扰不仅影响了信号的可读性，还可能导致特征提取和诊断结果的错误。本实验通过多级滤波设计，有效改善了数据质量，确保了后续分析的准确性。
4. **心电信号的医学意义**：心电信号处理不仅是工程领域的问题，还具有重要的医学应用价值。通过实验，我进一步认识到心率变异性（HRV）分析在心血管疾病早期筛查中的潜力。例如，RR 间隔的标准差能够反映心律的稳定性，而心率趋势则可用于评估短期内的心脏功能变化。
5. **科研能力的提升**：实验过程中，面对滤波器参数调整、动态门限优化等技术挑战，我通过不断调试与分析，逐步找到了合理的解决方案。这不仅提升了我的问题解决能力，也让我更加深刻地理解了科研中的耐心、细致和创新思维的重要性。

本次实验是对信号处理技术的一次全面实践，进一步加深了我对理论知识与实际应用结合的认识。同时，我也意识到生物医学信号处理在现代工程与医学中的巨大潜力。未来，我希望能够将这些技术应用到其他生物信号的处理与分析中，例如脑电（EEG）、肌电（EMG）等领域，不断扩展自己的知识广度与科研深度。

通过本次实验，我不仅加深了对心电信号处理的技术方法的理解，还提升了实践能力和科研素养，这将为我未来的学习与研究奠定坚实的基础。

五、实验代码

Listing 1: 心电数据的读取

```
file_path = '/Users/guhaoyu/Desktop/总文件夹/2024秋过期/
信号与系统/实验大作业/code/ECG_data.mat';
data = load(file_path);

% Step 2: 检查数据是否存在
if isfield(data, 'M')
    ecg_signal = data.M; % 读取 'M' 字段中的数据
else
    error('数据文件中未找到名为 "M" 的键，请检查数据文件。');
end

% Step 3: 检查数据的维度和内容
disp(['数据维度: ', mat2str(size(ecg_signal))]); % 打印数据维度
disp('数据示例:'); % 打印前5行
disp(ecg_signal(1:5, :));

% Step 4: 将数据保存为 CSV 文件
output_file = '/Users/guhaoyu/Desktop/总文件夹/2024秋过期/
信号与系统/实验大作业/code/ECG_signal.csv';
csvwrite(output_file, ecg_signal); % 保存为 CSV 文件
disp(['心电信号数据已保存为 ', output_file]);
```

Listing 2: 绘制 10s 双通道信号波形图

```
csv_file_path = '/Users/guhaoyu/Desktop/总文件夹/2024秋过期/
信号与系统/实验大作业/code/ECG_signal.csv';
data = readmatrix(csv_file_path); % 读取 CSV 文件，假设前两列是信号数据

% Step 2: 计算采样率
total_duration = 2 * 60; % 总时长为 2 分钟
num_samples = size(data, 1); % 数据点总数
sampling_rate = num_samples / total_duration; % 采样率
disp(['采样率为: ', num2str(sampling_rate), ' Hz']);

% Step 3: 生成时间轴
time_axis = linspace(0, total_duration, num_samples)'; % ...
    生成对应的时间点

% Step 4: 提取 90s 到 100s 的数据
```

```

start_time = 90; % 起始时间
end_time = 100; % 结束时间
start_index = floor(start_time * sampling_rate) + 1;
end_index = floor(end_time * sampling_rate);
time_segment = time_axis(start_index:end_index);
channel_1_segment = data(start_index:end_index, 1); % 第1通道
channel_2_segment = data(start_index:end_index, 2); % 第2通道

% Step 5: 绘制双通道信号波形图
figure;

% 绘制通道 1
plot(time_segment, channel_1_segment, 'b', 'DisplayName', 'Channel ...
    1'); % 通道 1 蓝色
hold on;

% 绘制通道 2
plot(time_segment, channel_2_segment, 'g', 'DisplayName', 'Channel ...
    2'); % 通道 2 绿色

% 图表美化
title('Dual-Channel ECG Signal (90s - 100s)', 'FontSize', 16);
xlabel('Time (s)', 'FontSize', 12);
ylabel('Amplitude', 'FontSize', 12);
legend;
grid on;

hold off;

```

Listing 3: 均值合并双通道信号

```

% Step 1: 用通道均值的方式合并为单通道信号
single_channel_signal = mean(data, 2); % ...
    计算每一行的均值，生成单通道信号

% Step 2: 绘制单通道信号波形图
figure;

% 绘制信号
plot(time_axis, single_channel_signal, 'g', 'DisplayName', 'Single ...
    Channel (Mean)'); % 绿色曲线

% 图表美化
title('Single-Channel ECG Signal (2 Minutes)', 'FontSize', 16); % ...

```



```

    设置标题
xlabel('Time (s)', 'FontSize', 12); % 设置横坐标标签
ylabel('Amplitude', 'FontSize', 12); % 设置纵坐标标签
legend; % 添加图例
grid on; % 显示网格

```

Listing 4: 绘制心电图信号的频谱特征

```

% Step 1: 计算单通道信号的频谱 (已计算的部分可复用)
N = length(single_channel_signal); % 信号长度
T = 1 / sampling_rate; % 采样周期
frequencies = (0:N/2-1) / (N * T); % 频率范围, 仅取前半部分
fft_values = fft(single_channel_signal); % FFT 计算
magnitude = abs(fft_values(1:N/2)); % 只取前半部分的幅值

% Step 2: 绘制 0.5–40 Hz 的频谱
% 仅保留 0.5–40 Hz 的频段
freq_range_1 = (frequencies ≥ 0.5) & (frequencies ≤ 40);
frequencies_zoomed_1 = frequencies(freq_range_1);
magnitude_zoomed_1 = magnitude(freq_range_1);

figure;

subplot(2, 1, 1); % 上部分
plot(frequencies_zoomed_1, magnitude_zoomed_1, 'g', 'DisplayName', ...
     '0.5–40 Hz Spectrum');
title('Frequency Spectrum (0.5–40 Hz)', 'FontSize', 16);
xlabel('Frequency (Hz)', 'FontSize', 12);
ylabel('Magnitude', 'FontSize', 12);
grid on;
legend;

% Step 3: 绘制 0.1–0.35 Hz 的低频基线漂移频谱
subplot(3, 1, 2);
freq_range_baseline = (frequencies ≥ 0.1) & (frequencies ≤ 0.35); % ...
    调整范围到 0.1–0.35 Hz
frequencies_zoomed_baseline = frequencies(freq_range_baseline);
magnitude_zoomed_baseline = magnitude(freq_range_baseline);
plot(frequencies_zoomed_baseline, magnitude_zoomed_baseline, 'r', ...
     'DisplayName', '0.1–0.35 Hz Baseline Drift');
title('Frequency Spectrum (0.1–0.35 Hz)', 'FontSize', 16);
xlabel('Frequency (Hz)', 'FontSize', 12);
ylabel('Magnitude', 'FontSize', 12);
grid on;

```

```

legend;

% Step 4: 绘制 30 Hz 至最大频率的频谱
% 仅保留 30 Hz 到最大可解析频率的频段
f_max = sampling_rate / 2; % 最大频率 (奈奎斯特频率)
freq_range_2 = (frequencies ≥ 30) & (frequencies ≤ f_max);
frequencies_zoomed_2 = frequencies(freq_range_2);
magnitude_zoomed_2 = magnitude(freq_range_2);

subplot(2, 1, 2); % 下部分
plot(frequencies_zoomed_2, magnitude_zoomed_2, 'b', 'DisplayName', '30 ...
      Hz to Nyquist Frequency Spectrum');
xlim([30, f_max]); % 横坐标从 30 开始到奈奎斯特频率
title(['Frequency Spectrum (30 Hz to ', num2str(f_max), ' Hz)'], ...
      'FontSize', 16);
xlabel('Frequency (Hz)', 'FontSize', 12);
ylabel('Magnitude', 'FontSize', 12);
grid on;
legend;

```

Listing 5: 滤波器的设计以及效果图绘制

```

% Step 1: 设计滤波器
fs = sampling_rate; % 采样频率
nyquist = fs / 2; % 奈奎斯特频率

% 1. 去除低频基线漂移 (高通滤波器, 截止频率 0.4 Hz)
[b_hp, a_hp] = butter(4, 0.4 / nyquist, 'high'); % 4 阶高通滤波器

% 2. 去除 60 Hz ± 5 Hz 工频干扰 (带阻滤波器, 频率范围 55–65 Hz)
[b_notch, a_notch] = butter(4, [55 65] / nyquist, 'stop'); % 4 ...
      阶带阻滤波器

% 3. 去除 100 Hz 以上的高频噪声 (低通滤波器, 截止频率 100 Hz)
[b_lp, a_lp] = butter(4, 100 / nyquist, 'low'); % 4 阶低通滤波器

% Step 2: 逐步滤波
signal_hp = filtfilt(b_hp, a_hp, single_channel_signal); % ...
      应用高通滤波器
signal_notch = filtfilt(b_notch, a_notch, signal_hp); % 应用带阻滤波器
signal_filtered = filtfilt(b_lp, a_lp, signal_notch); % 应用低通滤波器

% Step 3: 提取 5 秒时长的信号用于对比
time_axis = (0:length(single_channel_signal)-1) / fs; % 生成时间轴

```

```

start_time = 10; % 起始时间 (秒)
end_time = 15; % 结束时间 (秒)
segment_range = (time_axis ≥ start_time) & (time_axis ≤ end_time);

original_segment = single_channel_signal(segment_range);
filtered_segment = signal_filtered(segment_range);
time_segment = time_axis(segment_range);

% Step 4: 绘制去噪前后的波形对比图
figure;

% 原始信号
subplot(2, 1, 1);
plot(time_segment, original_segment, 'b', 'DisplayName', 'Original ...
    Signal');
title('Original Signal (5 Seconds)', 'FontSize', 16);
xlabel('Time (s)', 'FontSize', 12);
ylabel('Amplitude', 'FontSize', 12);
grid on;
legend;

% 去噪后的信号
subplot(2, 1, 2);
plot(time_segment, filtered_segment, 'r', 'DisplayName', 'Filtered ...
    Signal');
title('Filtered Signal (5 Seconds)', 'FontSize', 16);
xlabel('Time (s)', 'FontSize', 12);
ylabel('Amplitude', 'FontSize', 12);
grid on;
legend;

```

Listing 6: 心率的自动计算并绘制心率图

```

% Step 1: 检测 R 波峰值
[peaks, locs] = findpeaks(signal_filtered, 'MinPeakHeight', 0.5, ...
    'MinPeakDistance', 0.2 * fs); % 调整阈值

% Step 2: 计算 RR 间隔和心率
RR_intervals = diff(locs) / fs; % RR 间隔 (秒)
heart_rate = 60 ./ RR_intervals; % 心率 (bpm)
time_rr = locs(2:end) / fs; % RR 间隔对应的时间 (从第2个R波开始)

% Step 3: 绘制结果

```

```

figure;

% 绘制原始信号及 R 波
subplot(2, 1, 1);
plot(time_axis, signal_filtered, 'b', 'DisplayName', 'Filtered Signal');
hold on;
plot(locs / fs, peaks, 'ro', 'DisplayName', 'R Peaks');
title('ECG Signal with Detected R Peaks', 'FontSize', 16);
xlabel('Time (s)', 'FontSize', 12);
ylabel('Amplitude', 'FontSize', 12);
legend;
grid on;

% 在波形上标注心率值
for i = 1:6:length(heart_rate)
    text(time_rr(i), peaks(i+1), sprintf('%.1f bpm', heart_rate(i)), ...
        'Color', 'red', 'FontSize', 10, 'HorizontalAlignment', ...
        'center', 'VerticalAlignment', 'bottom');
end
hold off;

% 绘制心率随时间变化图
subplot(2, 1, 2);
plot(time_rr, heart_rate, '-o', 'Color', 'r', 'DisplayName', 'Heart ...
    Rate');
title('Heart Rate Trend', 'FontSize', 16);
xlabel('Time (s)', 'FontSize', 12);
ylabel('Heart Rate (bpm)', 'FontSize', 12);
grid on;
legend;

```